



حل پروژہی د گرسیون خطی از کلاس یاد گیری ماشین

گرد آورنده: امیر حسین ترنجیان

کلاس: یاد گیری ماشین

تابستان ۱۴۰۵

پیش‌بینی عملکرد تحصیلی دانش‌آموزان با استفاده از مدل‌های رگرسیون

خواندن مجموعه داده (Read the Dataset)

در اولین مرحله، مجموعه داده موردنظر با استفاده از کتابخانه Pandas در محیط پایتون بارگذاری شد. این مجموعه داده شامل اطلاعات مربوط به دانش‌آموزان از جنبه‌های مختلف فردی، خانوادگی، اجتماعی و آموزشی است. هدف از این مرحله آشنایی اولیه با ساختار داده‌ها، تعداد ویژگی‌ها و نوع اطلاعات موجود در هر ستون می‌باشد. همچنین با نمایش چند سطر ابتدایی داده‌ها، صحت بارگذاری فایل بررسی شد.

در اولین مرحله، داده‌های پروژه از فایل CSV خوانده شدند تا بتوان عملیات تحلیل و مدل‌سازی را روی آن انجام داد.

کد

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 9.9s

توضیح کد

import numpy as np

کتابخانه Numpy برای انجام محاسبات عددی و کار با آرایه‌های چندبعدی استفاده می‌شود.

import pandas as pd

کتابخانه Pandas برای خواندن، ذخیره‌سازی و پردازش داده‌های جدولی استفاده می‌شود.

import matplotlib.pyplot as plt

این کتابخانه برای رسم نمودارها و نمایش نتایج گرافیکی استفاده می‌شود.

`import seaborn as sns`

کتابخانه Seaborn برای رسم نمودارهای آماری و تحلیل داده‌ها استفاده می‌شود.

هدف

اولین مرحله در هر پروژه یادگیری ماشین، بارگذاری و بررسی اولیه داده‌ها است. در این مرحله داده‌ها از فایل CSV خوانده شده و ساختار کلی آن‌ها بررسی می‌شود.

مفهوم تئوری داده‌ها

داده‌ها مهم‌ترین بخش هر مدل یادگیری ماشین هستند. کیفیت داده مستقیماً روی عملکرد مدل تأثیر می‌گذارد. بنابراین قبل از هرگونه مدل‌سازی باید نوع ستون‌ها، تعداد رکوردها و وجود داده‌های ناقص بررسی شود.

```
df=pd.read_csv("C:/Intel/Students.csv")
df
```

✓ 0.1s Python

	Unnamed: 0	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	0	GP	F	18	U	GT3	A	4	4	at_home	...	4	3	4	1	1	3	6	5	6	6
1	1	GP	F	17	U	GT3	T	1	1	at_home	...	5	3	3	1	1	3	4	5	5	6
2	2	GP	F	15	U	LE3	T	1	1	at_home	...	4	3	2	2	3	3	10	7	8	10
3	3	GP	F	15	U	GT3	T	4	2	health	...	3	2	2	1	1	5	2	15	14	15
4	4	GP	F	16	U	GT3	T	3	3	other	...	4	3	2	1	2	5	4	6	10	10
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1039	1039	MS	F	19	R	GT3	T	2	3	services	...	5	4	2	1	2	5	4	10	11	10
1040	1040	MS	F	18	U	LE3	T	3	1	teacher	...	4	3	4	1	1	1	4	15	15	16
1041	1041	MS	F	18	U	GT3	T	1	1	other	...	1	1	1	1	1	5	6	11	12	9
1042	1042	MS	M	17	U	LE3	T	3	1	services	...	2	4	5	3	4	2	6	10	10	10
1043	1043	MS	M	18	R	LE3	T	3	2	services	...	4	4	1	3	4	5	4	10	11	11

1044 rows × 34 columns

تحلیل

در این مرحله تعداد نمونه‌ها و ویژگی‌های موجود در دیتاست بررسی شد.

➤ تعداد ردیف‌ها: ۱۰۴۴

ویژگی‌ها: ۳۴

نمایش و جایگزینی مقادیر گمشده

هدف

شناسایی و مدیریت داده‌های گمشده.

مفهوم تئوری

Missing Value یکی از رایج‌ترین مشکلات داده‌های واقعی است.

وجود مقادیر گمشده می‌تواند باعث:

- کاهش دقت مدل
- ایجاد خطا در آموزش
- ایجاد Bias در نتایج

شود.

روش‌های رایج:

- حذف رکورد

- جایگزینی با میانگین
- جایگزینی با میانه
- جایگزینی با مد

کد

```
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1044 entries, 0 to 1043
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   1044 non-null   int64
1   school       1044 non-null   object
2   sex          1044 non-null   object
3   age          1044 non-null   int64
4   address      1044 non-null   object
5   famsize      1044 non-null   object
6   Pstatus      1044 non-null   object
7   Medu         1044 non-null   int64
8   Fedu         1044 non-null   int64
9   Mjob         1044 non-null   object
10  Fjob         1044 non-null   object
11  reason       1044 non-null   object
12  guardian     1044 non-null   object
13  traveltime   1044 non-null   int64
14  studytime    1044 non-null   int64
15  failures     1044 non-null   int64
16  schoolsup     1044 non-null   object
17  famsup       1044 non-null   object
18  paid         1044 non-null   object
19  activities   1044 non-null   object
...
32  G2           1044 non-null   int64
33  G3           1044 non-null   int64
dtypes: int64(17), object(17)
memory usage: 277.4+ KB
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

نتایج حاصل از این کد:

این دستور اطلاعات زیر را نمایش می دهد:

- تعداد رکوردها
- نوع داده هر ستون
- تعداد مقادیر غیر Null

```
df.isnull().sum()
✓ 0.0s

Unnamed: 0    0
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
...
absences      0
G1            0
G2            0
G3            0
dtype: int64

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

توضیح خط به خط

isnull()

خانه‌های خالی را شناسایی می‌کند.

sum()

تعداد آن‌ها را محاسبه می‌کند.

تحلیل

در این پروژه مشاهده شد که هیچ داده گمشده وجود نداشت.

بررسی توزیع داده‌ها

کد

```
df['sex'].value_counts()
✓ 0.0s

sex
F    591
M    453
Name: count, dtype: int64

df['age'].value_counts()
✓ 0.0s

age
16    281
17    277
18    222
15    194
19     56
20     9
21     3
22     2
Name: count, dtype: int64
```

```
df['G3'].value_counts().sort_values()
✓ 0.0s

G3
1      1
20     1
4      1
19     7
5      8
6     18
7     19
18     27
17     35
16     52
0      53
9      63
8      67
15     82
14     90
12    103
13    113
11    151
10    153
Name: count, dtype: int64
```

این دستورات تعداد وقوع هر مقدار را در ستون‌های مختلف محاسبه می‌کنند.

اطلاعات بیشتر در مورد دیتاست:

```
df.describe().transpose()
```

✓ 0.1s

	count	mean	std	min	25%	50%	75%	max
Unnamed: 0	1044.0	521.500000	301.521144	0.0	260.75	521.5	782.25	1043.0
age	1044.0	16.726054	1.239975	15.0	16.00	17.0	18.00	22.0
Medu	1044.0	2.603448	1.124907	0.0	2.00	3.0	4.00	4.0
Fedu	1044.0	2.387931	1.099938	0.0	1.00	2.0	3.00	4.0
traveltime	1044.0	1.522989	0.731727	1.0	1.00	1.0	2.00	4.0
studytime	1044.0	1.970307	0.834353	1.0	1.00	2.0	2.00	4.0
failures	1044.0	0.264368	0.656142	0.0	0.00	0.0	0.00	3.0
famrel	1044.0	3.935824	0.933401	1.0	4.00	4.0	5.00	5.0
freetime	1044.0	3.201149	1.031507	1.0	3.00	3.0	4.00	5.0
goout	1044.0	3.156130	1.152575	1.0	2.00	3.0	4.00	5.0
Dalc	1044.0	1.494253	0.911714	1.0	1.00	1.0	2.00	5.0
Walc	1044.0	2.284483	1.285105	1.0	1.00	2.0	3.00	5.0
health	1044.0	3.543103	1.424703	1.0	3.00	4.0	5.00	5.0
absences	1044.0	4.434866	6.210017	0.0	0.00	2.0	6.00	75.0
G1	1044.0	11.213602	2.983394	0.0	9.00	11.0	13.00	19.0
G2	1044.0	11.246169	3.285071	0.0	9.00	11.0	13.00	19.0
G3	1044.0	11.341954	3.864796	0.0	10.00	11.0	14.00	20.0

با کد فوق شاخص‌های آماری زیر محاسبه می‌شوند:

- میانگین
- انحراف معیار
- حداقل
- حداکثر
- Quartiles

شناسایی مقادیر اشتباه

هدف

شناسایی داده‌های غیرمنطقی و پرت.

مفهوم تئوری

گاهی داده‌ها از نظر آماری معتبر هستند اما از نظر منطقی اشتباه‌اند.

مثلاً:

- سن منفی
- نمره خارج از بازه
- تعداد غیبت غیرواقعی

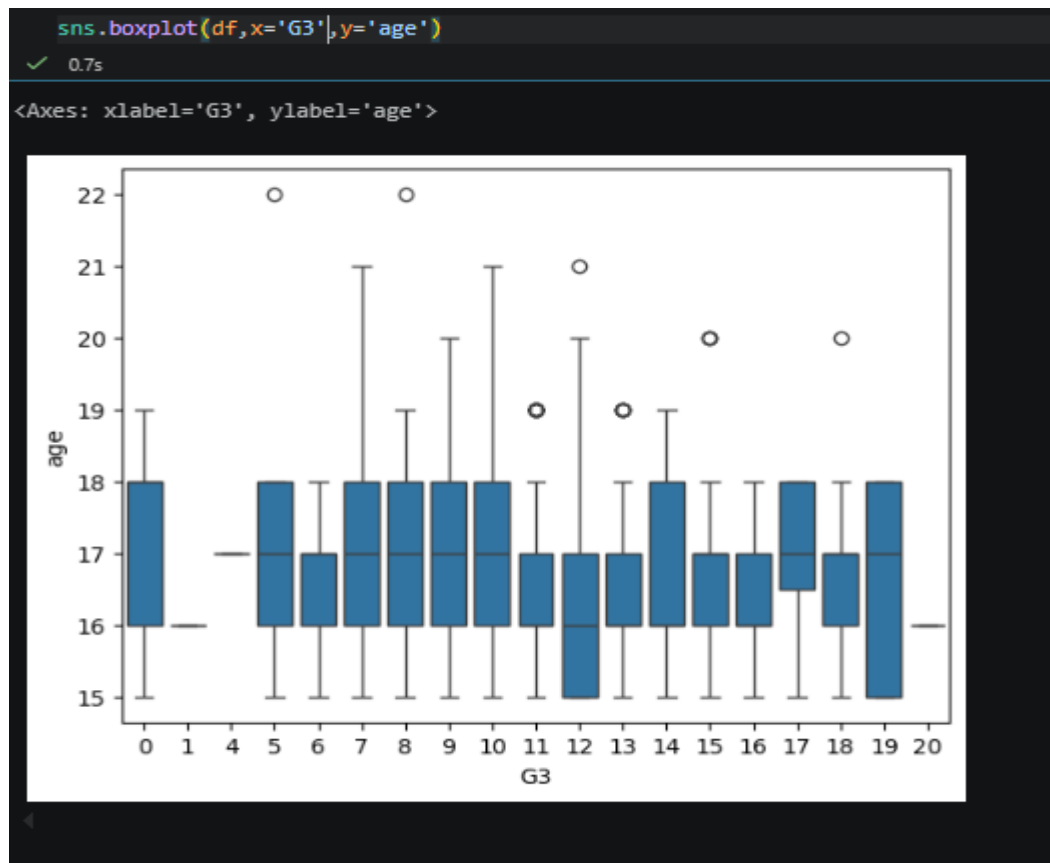
به این داده‌ها Wrong Value گفته می‌شود

مفهوم Boxplot

Boxplot ابزاری برای شناسایی Outlier است.

اجزای آن:

- Median
- Quartile اول
- Quartile سوم
- نقاط پرت



همچنین ممکن است داده پرت (Outlier) وجود داشته باشد که روی مدل اثر منفی بگذارد.

کد

```
print(df[(df["age"] < 15) | (df["age"] > 22)])
```

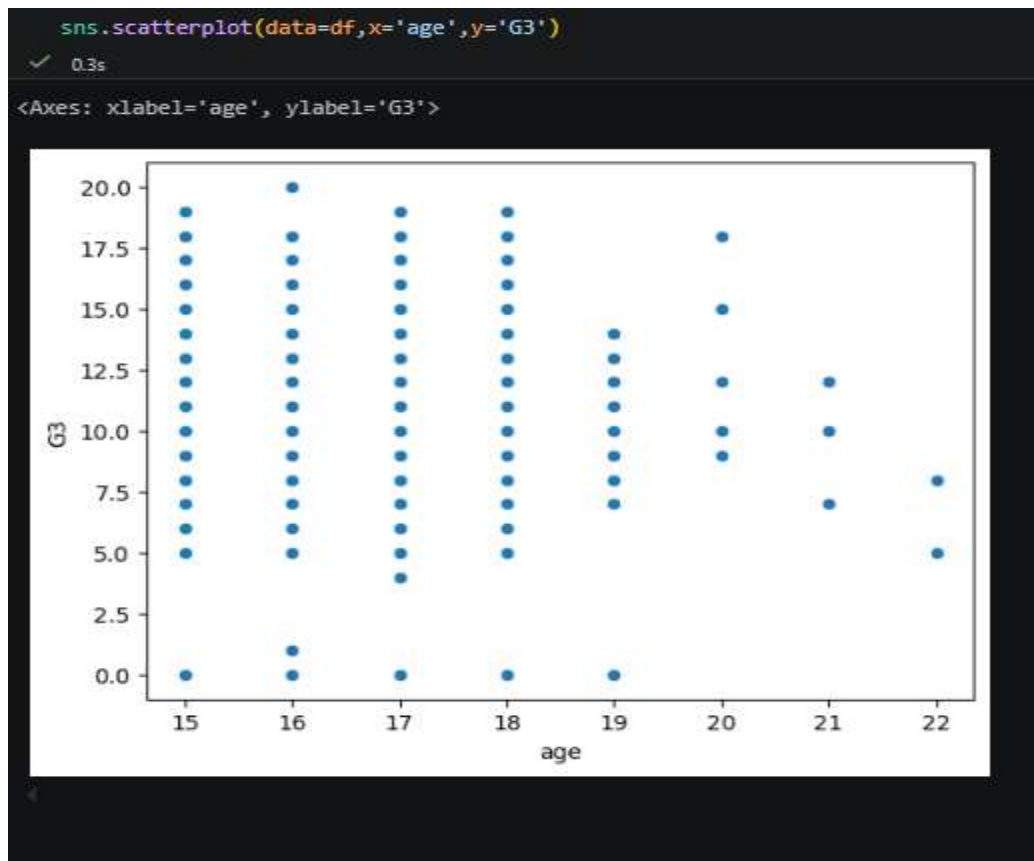
✓ 0.0s

Empty DataFrame
Columns: [Unnamed: 0, school, sex, age, address, famsize, Pstatus, Medu, Fedu, Mjob,
Index: []
[0 rows x 34 columns]

توضیح

داده‌های پرت در ستون سن دانش آموزان وجود ندارد.

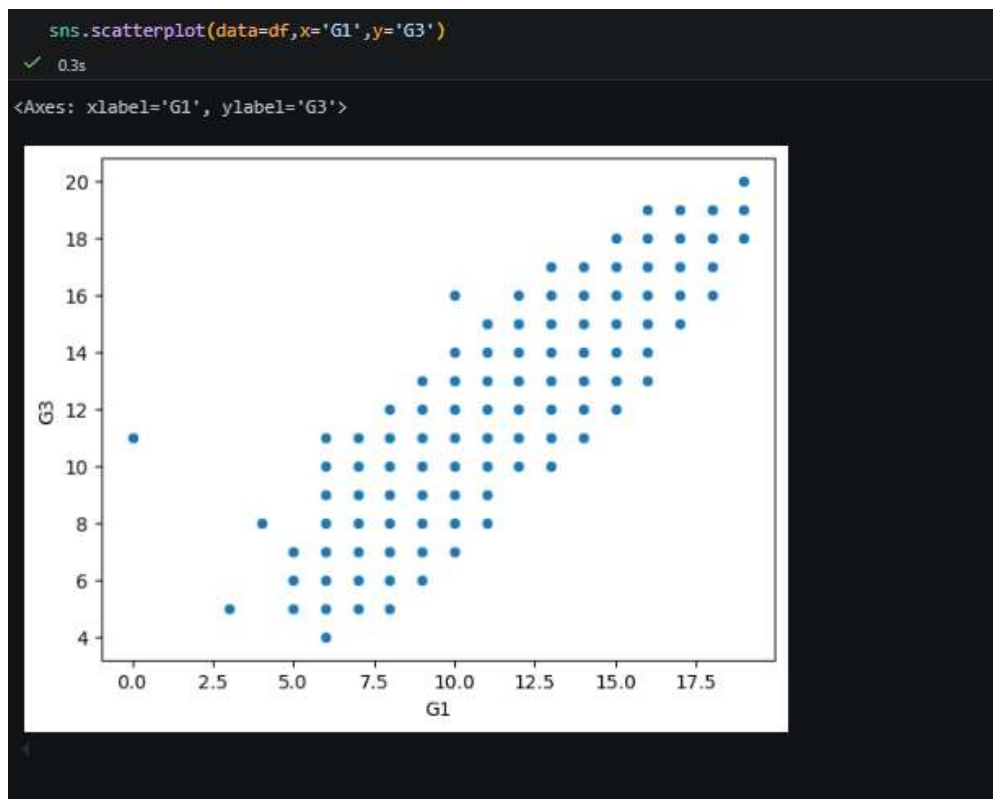
نمایش داده‌های پرت



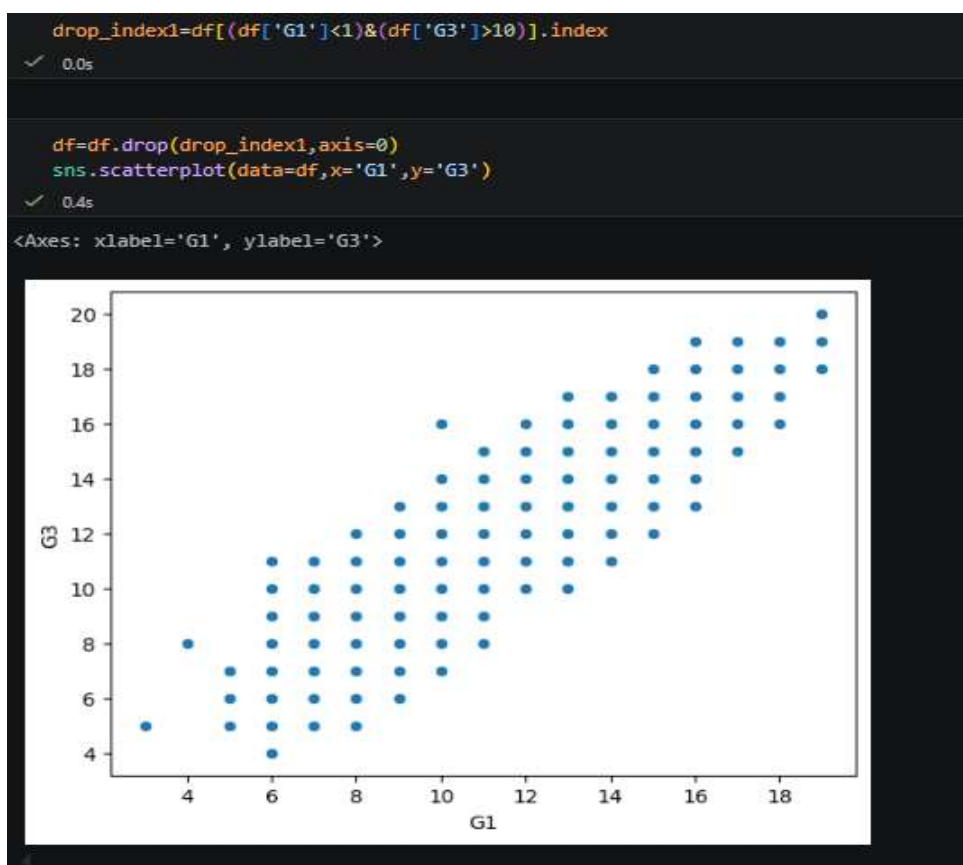
حذف داده‌های پرت



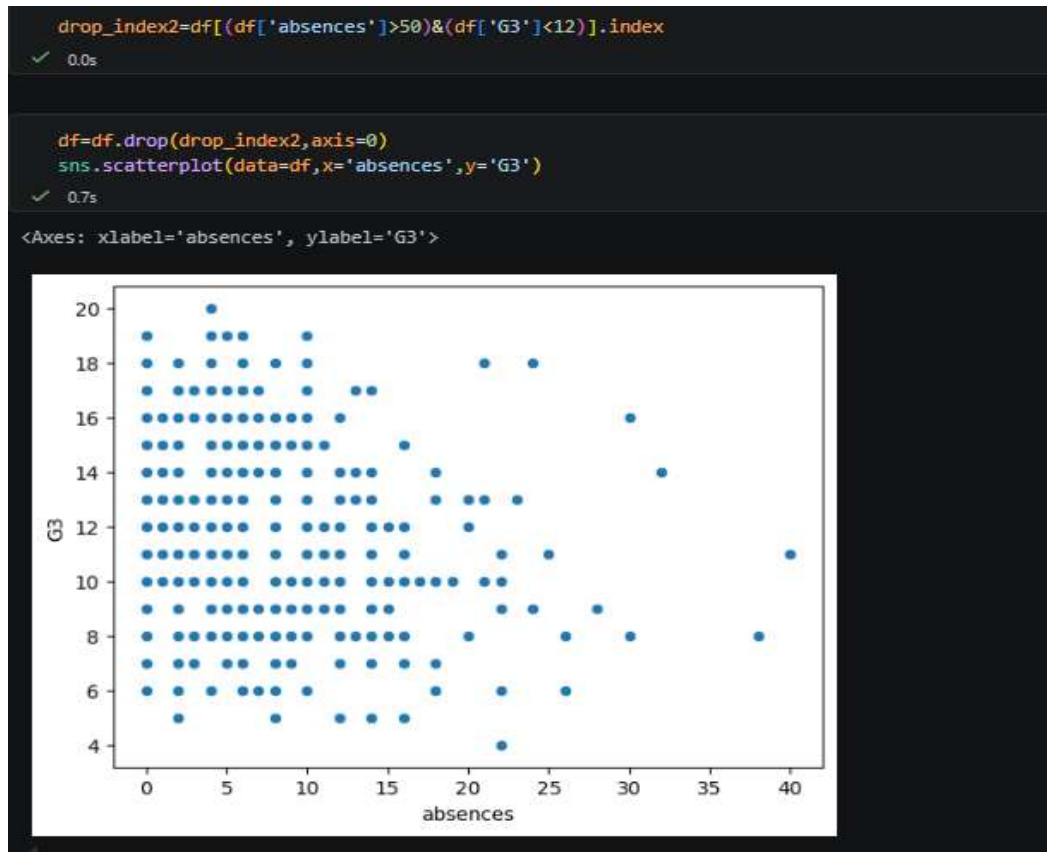
برخی دیگر از حذف داده‌های پرت



کد



برخی دیگر از حذف داده‌های پرت



تبدیل objects به داده‌های عددی

هدف

تبدیل داده‌های متنی به داده‌های عددی.

مفهوم تئوری

الگوریتم‌های یادگیری ماشین که به آن در پروژه اشاره شده فقط با داده‌های عددی کار می‌کنند.

بنابراین ویژگی‌هایی مانند:

sex
school
address

باید به عدد تبدیل شوند.

✓ روش‌های رایج: Label Encoding

هر دسته یک عدد می‌گیرد.

مثال:

Male = 0
Female = 1

ایراد آن: ارزش‌گذاری می‌شود

✓ One-Hot Encoding

برای هر دسته یک ستون جداگانه ساخته می‌شود.

مثال:

school_GP
school_MS

• جلوگیری از ایجاد ترتیب مصنوعی بین دسته‌ها

مزیت

• احتمال Overfitting افزایش می‌یابد.

• افزایش هزینه‌ی محاسبات

ایراد

الگوریتم‌های یادگیری ماشین با داده‌های متنی کار نمی‌کنند؛ بنابراین داده‌های متنی باید به داده‌های عددی تبدیل شوند و تمام ستون‌های متنی استخراج می‌شوند.

کد

```
object_df=df.select_dtypes(include=object)
```

✓ 0.0s

object_df

✓ 0.0s

	school	sex	address	famsize	Pstatus	Mjob	Fjob	reason	guardian	schoolsup	famsup	paid	activities	nursery	higher	internet	romantic
0	GP	F	U	GT3	A	at_home	teacher	course	mother	yes	no	no	no	yes	yes	no	no
1	GP	F	U	GT3	T	at_home	other	course	father	no	yes	no	no	no	yes	yes	no
2	GP	F	U	LE3	T	at_home	other	other	mother	yes	no	yes	no	yes	yes	yes	no
3	GP	F	U	GT3	T	health	services	home	mother	no	yes	yes	yes	yes	yes	yes	yes
4	GP	F	U	GT3	T	other	other	home	father	no	yes	yes	no	yes	yes	no	no
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1039	MS	F	R	GT3	T	services	other	course	mother	no	no	no	yes	no	yes	yes	no
1040	MS	F	U	LE3	T	teacher	services	course	mother	no	yes	no	no	yes	yes	yes	no
1041	MS	F	U	GT3	T	other	other	course	mother	no	no	no	yes	yes	yes	no	no
1042	MS	M	U	LE3	T	services	services	course	mother	no	no	no	no	no	yes	yes	no
1043	MS	M	R	LE3	T	services	other	course	mother	no	no	no	no	no	yes	yes	no

986 rows x 17 columns

توضیح

تابع `get_dummies` عملیات One-Hot Encoding را انجام می‌دهد.

کد

```
object_df_dummies=pd.get_dummies(object_df,drop_first=True)
```

object_df_dummies

✓ 0.1s

Python

	school_MS	sex_M	address_U	famsize_LE3	Pstatus_T	Mjob_health	Mjob_other	Mjob_services	Mjob_teacher	Fjob_health	...	guardian_mother	guardian_other	schoolsup_yes
0	False	False	True	False	False	False	False	False	False	False	...	True	False	True
1	False	False	True	False	True	False	False	False	False	False	...	False	False	False
2	False	False	True	True	True	False	False	False	False	False	...	True	False	True
3	False	False	True	False	True	True	False	False	False	False	...	True	False	False
4	False	False	True	False	True	False	True	False	False	False	...	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1039	True	False	False	False	True	False	False	True	False	False	...	True	False	False
1040	True	False	True	True	True	False	False	False	True	False	...	True	False	False
1041	True	False	True	False	True	False	True	False	False	False	...	True	False	False
1042	True	True	True	True	True	False	False	True	False	False	...	True	False	False
1043	True	True	False	True	True	False	False	True	False	False	...	True	False	False

986 rows x 26 columns

برای تبدیل متغیرهای دسته‌ای از One-Hot Encoding استفاده شده است. برای جلوگیری از Dummy Variable Trap با استفاده از drop_first یک ستون حذف می‌شود.

کد

```
numeric_df=df.select_dtypes(exclude=object)
final_df=pd.concat([numeric_df,object_df_dummies],axis=1)
```

✓ 0.0s

Generate

Code

Markdown

Python

```
final_df
```

✓ 0.0s

Python

	Unnamed: 0	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	_	guardian_mother	guardian_other	schoolsup_yes	famsup_yes	paid_yes	activities_yes
0	0	18	4	4	2	2	0	4	3	4	_	True	False	True	False	False	False
1	1	17	1	1	1	2	0	5	3	3	_	False	False	False	True	False	False
2	2	15	1	1	1	2	3	4	3	2	_	True	False	True	False	True	False
3	3	15	4	2	1	3	0	3	2	2	_	True	False	False	True	True	True
4	4	16	3	3	1	2	0	4	3	2	_	False	False	False	True	True	False
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
1039	1039	19	2	3	1	3	1	5	4	2	_	True	False	False	False	False	True
1040	1040	18	3	1	1	2	0	4	3	4	_	True	False	False	True	False	False
1041	1041	18	1	1	2	2	0	1	1	1	_	True	False	False	False	False	True
1042	1042	17	3	1	2	1	0	2	4	5	_	True	False	False	False	False	False
1043	1043	18	3	2	3	1	0	4	4	1	_	True	False	False	False	False	False

986 rows x 43 columns

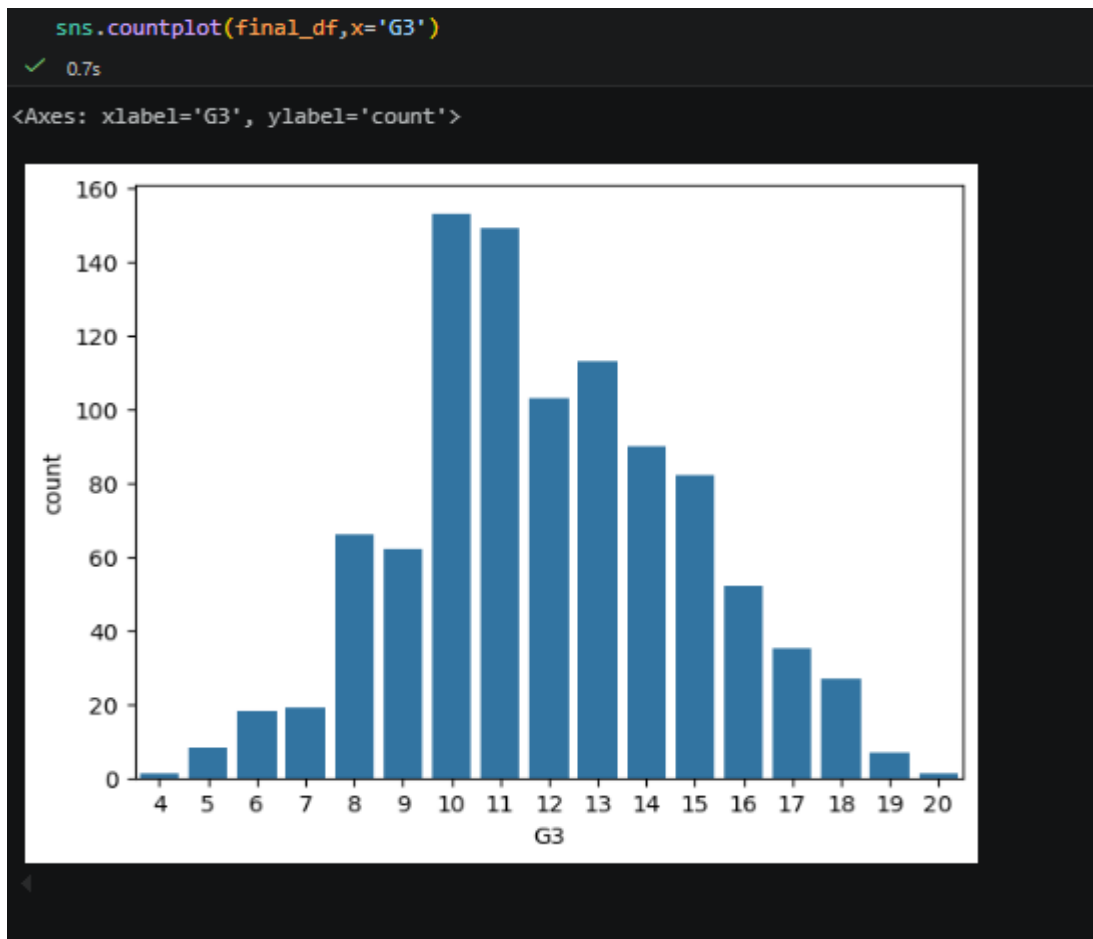
داده‌های عددی و داده‌های تبدیل شده با هم ادغام می‌شوند.

مفهوم تنوری

قبل از آموزش مدل باید توزیع متغیر هدف بررسی شود. برای بررسی توزیع داده‌ها از نمودار CountPlot استفاده شده است.

اگر داده‌ها نامتوازن باشند، عملکرد مدل کاهش می‌یابد.

کد



ارتباط بین ویژگی‌ها و ستون‌ها

هدف

بررسی ارتباط ویژگی‌ها با متغیر هدف.

ارتباط بین داده‌ها

```
sns.pairplot(final_df[['absences', 'age', 'G1', 'G2', 'G3', 'schoolsup_yes', 'guardian_mother']])
```

✓ 14.9s

Heatmap روابط بین تمامی ویژگی‌ها را نمایش می‌دهد.

ارتباط مستقیم	ارتباط معکوس	بدون ارتباط
کوچک تر از صفر	بزرگ تر از صفر	مساوی صفر

کد

```
plt.figure(figsize=(40,50),dpi=200)
sns.heatmap(final_df.corr(),annot=True)
```

✓ 12.9s

شدت رنگ نشان‌دهنده میزان همبستگی است.

تعیین ورودی و خروجی

مفهوم

در یادگیری نظارت‌شده:

- X شامل ویژگی‌های ورودی است.
- Y شامل نمره نهایی دانش‌آموزان (G3) است.

کد

```
x=final_df.drop('G3',axis=1)
y=final_df['G3']
✓ 0.0s
```

80 درصد داده‌ها برای آموزش و 20 درصد برای آزمون استفاده شدند.

کد

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=101)
✓ 0.8s
```

فرآیند Polynomial Preprocessing

هدف

ویژگی‌های جدیدی از ویژگی‌های اصلی ایجاد می‌کند و باعث ایجاد روابط غیرخطی بین ویژگی‌ها می‌شود.

```
poly_converter=PolynomialFeatures(degree=d,include_bias=False)
```

در PolynomialFeatures پارامتر include_bias مشخص می‌کند که ستون ثابت 1 به داده‌ها اضافه شود یا نه.

مثلاً اگر داشته باشیم:

$$X = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

و بنویسیم:

```
poly = PolynomialFeatures(  
    degree=2,  
    include_bias=True  
)
```

خروجی:

	x	x^2
1	2	4
1	3	9

یعنی ستون اول همه جا 1 است.

اما اگر:

```
poly = PolynomialFeatures(  
    degree=2,  
    include_bias=False  
)
```

خروجی:

	x	x^2
2	4	
3	9	

دیگر ستون ۱ اضافه نمی شود.

این ستون ۱ برای چیست؟

این ستون متناظر با عرض از مبدأ (Intercept) در معادله رگرسیون است:

$$y = b_0 + b_1x + b_2x^2$$

عدد ۱ در واقع ضریب (b_0) را وارد مدل می‌کند.

در Scikit-Learn معمولاً چه کار می‌کنیم؟

اکثر مدل‌ها مثل:

LinearRegression()

Ridge()

Lasso()

ElasticNet()

خودشان Intercept را یاد می‌گیرند:

fit_intercept=True

پس معمولاً می‌نویسیم:

```
poly = PolynomialFeatures(  
    degree=2,  
    include_bias=False  
)
```

تا ستون اضافی و تکراری ساخته نشود.

مفهوم تنوری

Polynomial Features ویژگی‌های جدیدی تولید می‌کند.

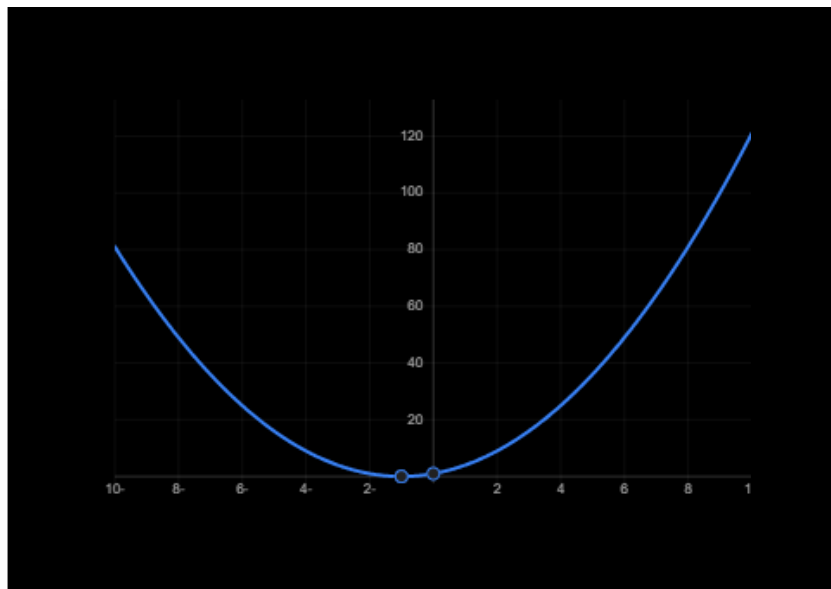
فرمول Polynomial Regression در واقع گسترش یافته‌ی Linear Regression است.

$$\hat{y} = b_0 + b_1x$$

اما در Polynomial Regression جمله‌های توان‌دار به مدل اضافه می‌شوند:

$$\hat{y} = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$$

مثال: برای Degree=2 داریم



این مدل می‌تواند روابط خمیده (غیرخطی) را یاد بگیرد.

برای چند ویژگی

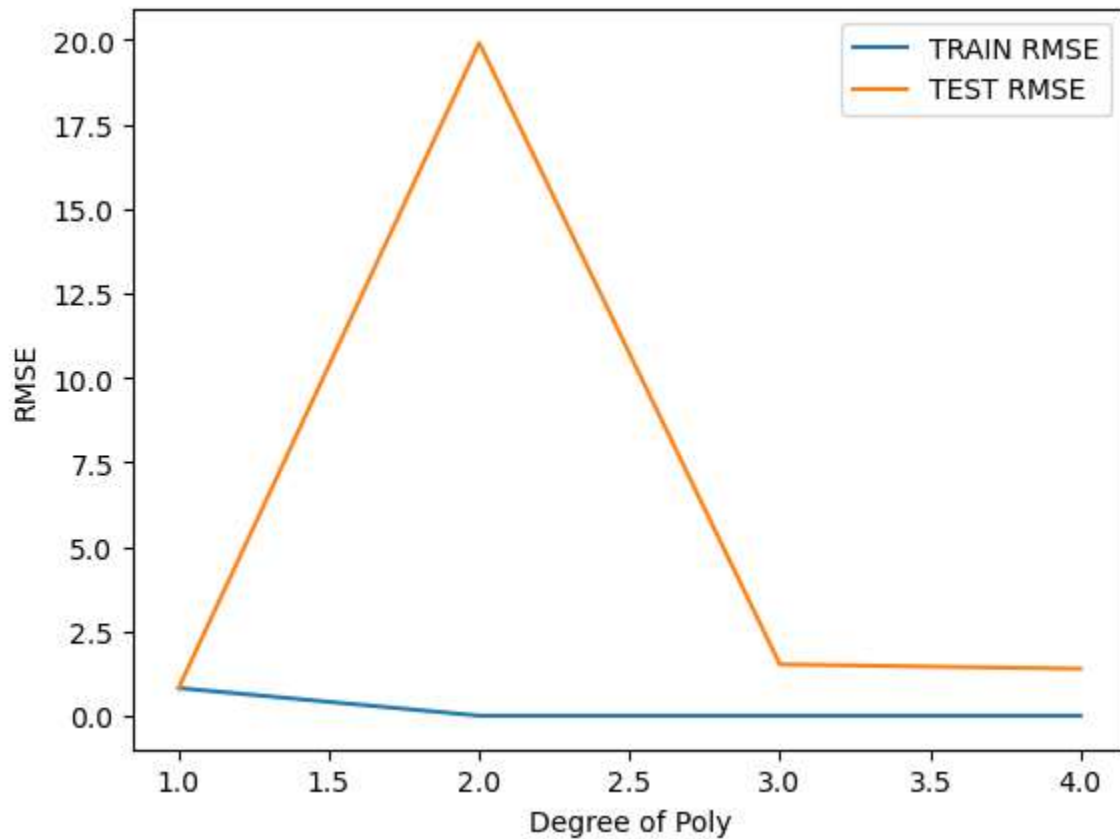
اگر دو ویژگی داشته باشیم:

- x_1
- x_2

و درجه ۲ انتخاب کنیم:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_1^2 + b_4x_2^2 + b_5x_1^2$$

در این پروژه از درجه یک استفاده شده چون خطاش روی مدل ها کمتره برای مثال در LinearRegression درجه یک مناسب تر است.



در تست درجه یک از همه کمتر است به مرور با افزایش درجه پایین تر می آید ولی از چهار به بعد هزینه‌ی محاسبات به شدت بالا می آید.

آموزش و پیش‌بینی با الگوریتم‌های رگرسیون خطی

Linear Regression

مفهوم

رگرسیون خطی یک روش یادگیری نظارت‌شده برای پیش‌بینی متغیرهای پیوسته است. این مدل رابطه بین متغیر هدف و ویژگی‌های ورودی را به صورت یک ترکیب خطی مدل‌سازی می‌کند.

فرمول:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots + b_x x_x$$

یا به صورت کلی:

$$\hat{y} = b_0 + \sum_{i=1}^n b_i x_i$$

مزایا

- سریع
- قابل تفسیر

معایب

- ناتوان در مدل‌سازی روابط پیچیده

تابع خطا (Cost Function)

Linear Regression ضرایب را طوری پیدا می کند که مجموع خطاها کمینه شود.

معمولاً از **Mean Squared Error (MSE)** استفاده می شود:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

که در آن:

y_i مقادیر واقعی

$\hat{y}_i$ مقادیر پیش بینی شده

کد

```
model=LinearRegression()
model.fit(poly_features,y_train)
✓ 0.0s
```

مدل رگرسیون خطی روی داده های آموزشی، آموزش داده شد.

Mean Absolute Error (MAE)

MAE میانگین قدر مطلق اختلاف بین مقدار واقعی و مقدار پیش بینی شده توسط مدل را محاسبه می کند.

به زبان ساده:

این معیار نشان می‌دهد مدل به طور متوسط چقدر خطا دارد.

برای مثال اگر MAE برابر 1.5 باشد، یعنی پیش‌بینی‌های مدل به طور میانگین حدود ۱,۵ واحد با مقادیر واقعی اختلاف دارند.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

که در آن:

- n : تعداد نمونه‌ها
- y_i : مقدار واقعی
- $\hat{y}_i$: مقدار پیش‌بینی شده
- $|y_i - \hat{y}_i|$: قدر مطلق خطا

مزایا

✓ فهم و تفسیر بسیار آسان

✓ حساسیت کمتر به داده‌های پرت (Outlier)

✓ واحد خروجی با واحد متغیر هدف یکسان است

معایب

✗ خطاهای بزرگ را به اندازه کافی جریمه نمی‌کند

✗ تفاوت بین خطای ۲ و ۲۰ را خیلی شدید نشان نمی‌دهد

"معیار MAE میانگین مقدار مطلق خطاهای پیش‌بینی را اندازه‌گیری می‌کند. این معیار نشان می‌دهد که مدل به طور متوسط چه میزان از مقدار واقعی فاصله دارد. هرچه مقدار MAE کوچک‌تر باشد، عملکرد مدل بهتر خواهد بود."

MSE میانگین مربع اختلاف بین مقادیر واقعی و پیش‌بینی شده را محاسبه می‌کند.

برخلاف MAE، در این معیار خطاها به توان دو می‌رسند.

فرمول MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

مزایا

✓ خطاهای بزرگ را شدیداً جریمه می‌کند

✓ مناسب برای بهینه‌سازی مدل‌های یادگیری ماشین

✓ معیار استاندارد در بسیاری از الگوریتم‌ها

معایب

✗ نسبت به داده‌های پرت بسیار حساس است

✗ تفسیر آن از MAE سخت‌تر است

✗ واحد آن مربع واحد متغیر هدف است

```

poly_converter=PolynomialFeatures(degree=1,include_bias=False)
poly_features=poly_converter.fit_transform(x_train)

x_test=poly_converter.transform(x_test)
scaler=StandardScaler()
poly_features=scaler.fit_transform(poly_features)
x_test=scaler.transform(x_test)
model=LinearRegression()
model.fit(poly_features,y_train)
y_pred=model.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)

```

MAE : 0.6498278580065135

MSE : 0.6685966251275709

RMSE : 0.8176775801791137

✓ در گزارش بعدی به تفصیل به *StandardScaler* پرداخته خواهد شد.

SVR(Support Vector Machine)

مفهوم SVR

الگوریتم SVR نسخه رگرسیونی ماشین بردار پشتیبان (SVM) است. هدف این الگوریتم یافتن تابعی است که با کمترین خطا داده‌ها را مدل‌سازی کند. برخلاف رگرسیون خطی که تنها به کمینه کردن خطا توجه دارد، SVR تلاش می‌کند داده‌ها را در یک ناحیه خطای قابل قبول قرار دهد و نسبت به داده‌های پرت مقاومت بیشتری داشته باشد.

تابع پیش‌بینی در SVR به صورت زیر بیان می‌شود:

$$f(x) = \omega^T x + b$$

که در آن ω بردار ضرایب و b عرض از مبدأ است.

هدف آن پیدا کردن بهترین تابعی است که بیشترین تعداد نقاط را درون یک ناحیه خطا قرار دهد.

مزایا

- مناسب داده‌های پیچیده
- مقاومت بالا در برابر نویز

پارامترهای مهم

C

کنترل میزان خطا

- C بزرگ $\rightarrow$ دقت بیشتر
- C کوچک $\rightarrow$ تعمیم بهتر

ϵ

عرض ناحیه مجاز خطا

- ϵ بزرگ $\rightarrow$ مدل ساده‌تر
- ϵ کوچک $\rightarrow$ مدل پیچیده‌تر

Kernel

برای مدل‌سازی روابط غیرخطی

تابع هزینه‌ی SVR

$$\frac{1}{2} \|\omega\|^2 + c \sum (\xi_I - \xi_I^*)$$

به شرط:

$$\begin{aligned} y_i - f(x_i) &\leq \epsilon + \xi_i \\ f(x_i) - y_i &\leq \epsilon + \xi_i^* \end{aligned}$$

کد

با استفاده از این کد بهترین درجه را پیدا می‌کنیم.

```
from sklearn.preprocessing import StandardScaler
train_rmse_errors=[]
test_rmse_errors=[]
for d in range(1,5):
    x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=101)
    poly_converter=PolynomialFeatures(degree=d,include_bias=False)
    poly_features=poly_converter.fit_transform(x_train)
    x_test=poly_converter.transform(x_test)
    scaler=StandardScaler()
    poly_features=scaler.fit_transform(poly_features)
    x_test=scaler.transform(x_test)

    model=SVR()
    model.fit(poly_features,y_train)

    train_pred=model.predict(poly_features)
    test_pred=model.predict(x_test)

    train_mse=mean_squared_error(y_train,train_pred)
    train_rmse=np.sqrt(train_mse)

    test_mse=mean_squared_error(y_test,test_pred)
    test_rmse=np.sqrt(test_mse)

    train_rmse_errors.append(train_rmse)
```

```

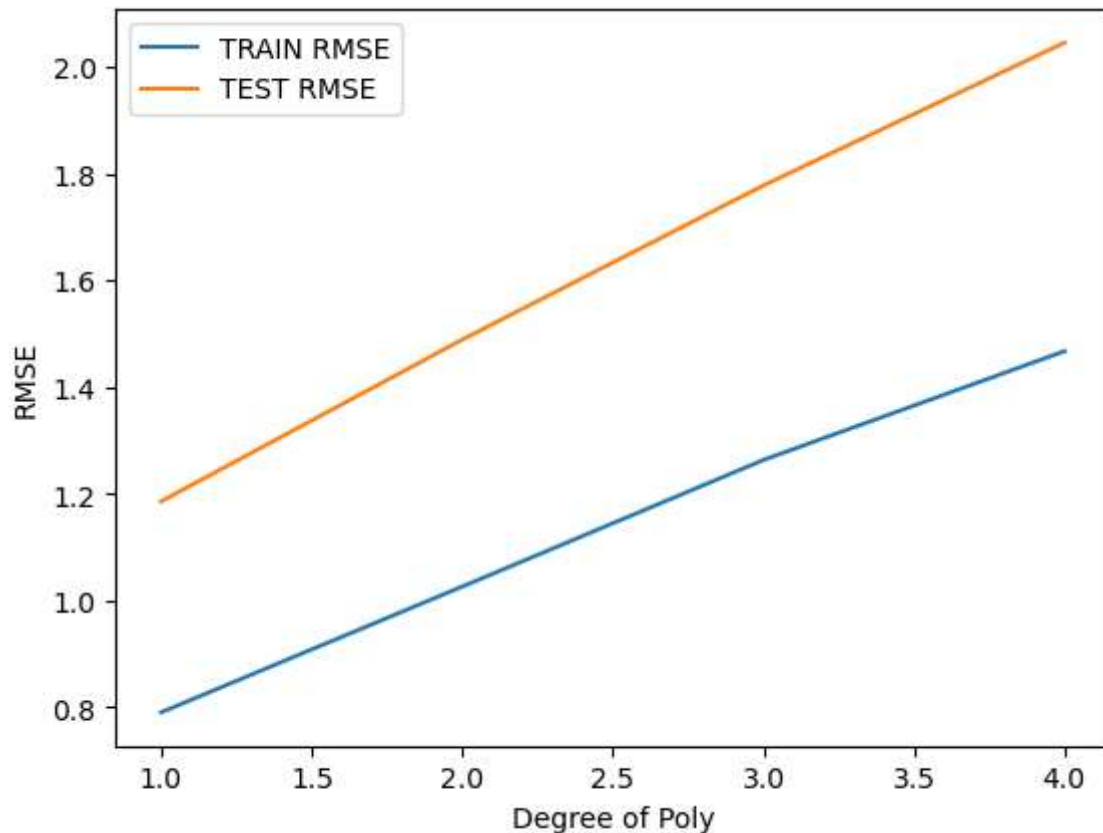
test_rmse_errors.append(test_rmse)
plt.plot(range(1,5),train_rmse_errors[:5],label='TRAIN RMSE')
plt.plot(range(1,5),test_rmse_errors[:5],label='TEST RMSE')

plt.xlabel('Degree of Poly')
plt.ylabel('RMSE')

```

تحلیل

نمودار نشان می‌دهد که کمترین خطا رو درجه یک می‌دهد.



GridSearchCv

```

from sklearn.model_selection import GridSearchCV
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=101)
poly_converter=PolynomialFeatures(degree=1,include_bias=False)
poly_features=poly_converter.fit_transform(x_train)
x_test=poly_converter.transform(x_test)
scaler=StandardScaler()
poly_features=scaler.fit_transform(poly_features)

```

```
x_test=scaler.transform(x_test)
model=SVR()
params_grid={'C':[0.01,0.05,0.1,1,10,100,1000],
'kernel':['linear','rbf','sigmoid','poly'],'degree':[1,10]}
grid_model=GridSearchCV(model,params_grid,cv=5,scoring='neg_mean_squared_error')
grid_model.fit(poly_features,y_train)
y_pred=grid_model.predict(x_test)
```

بهترین پارامترهای مدل SVR توسط Grid Search پیدا شدند.

پارامترهای بررسی شده:

- C •
- kernel •
- degree •

```
grid_model.best_params_
{'C': 0.1, 'degree': 1, 'kernel': 'linear'}
```

کد نتایج ارزیابی خطا:

```
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
```

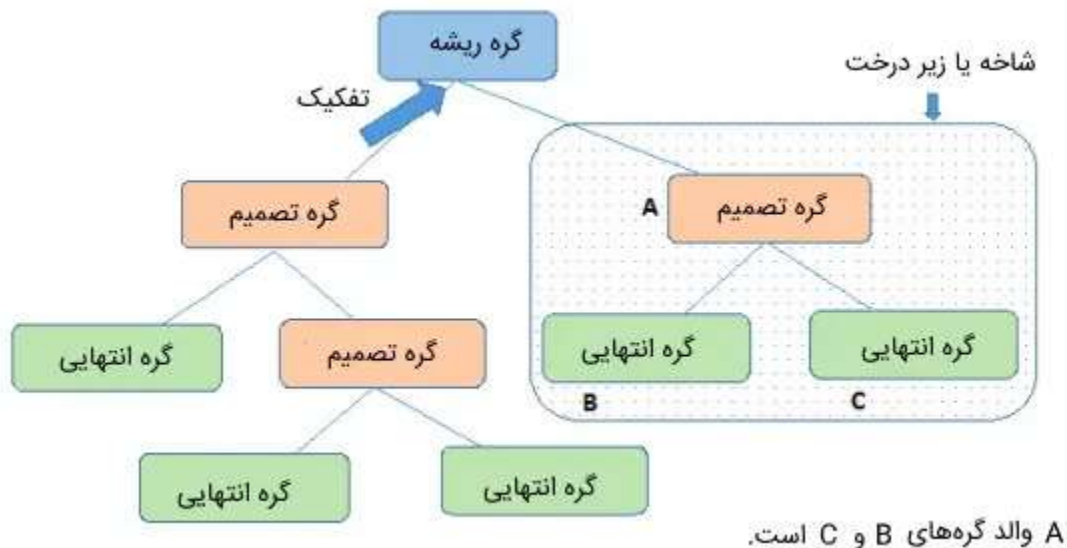
```
MAE : 0.6741971261717687
MSE : 0.7355816162563278
RMSE : 0.8576605483851567
```

Decision Tree Regressor

مفهوم Decision Tree Regressor

داده‌ها را به صورت سلسله مراتبی تقسیم می‌کند. در هر مرحله بهترین ویژگی برای تقسیم انتخاب می‌شود. درخت تصمیم رگرسیونی (Decision Tree Regressor) یکی از الگوریتم‌های یادگیری نظارت‌شده است که برای پیش‌بینی مقادیر پیوسته استفاده می‌شود. این الگوریتم با تقسیم مکرر داده‌ها به زیرمجموعه‌های کوچک‌تر، روابط بین متغیرهای ورودی و خروجی را یاد می‌گیرد.

برخلاف رگرسیون خطی که یک رابطه ریاضی مشخص بین ورودی و خروجی فرض می‌کند، درخت تصمیم هیچ فرضی درباره شکل رابطه داده‌ها ندارد و می‌تواند الگوهای غیرخطی را نیز مدل‌سازی کند.



درخت از سه بخش اصلی تشکیل شده است:

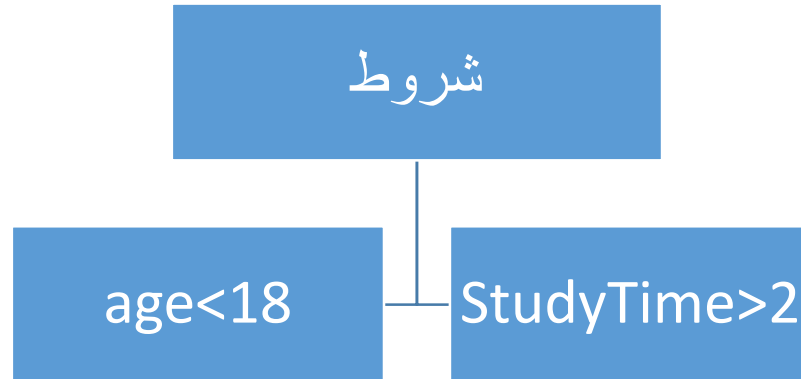
گره ریشه

اولین گره درخت که کل داده‌ها را در اختیار دارد.

گره‌های داخلی

در هر گره یک شرط روی یکی از ویژگی‌ها اعمال می‌شود.

مثال:



برگ‌ها

گره‌های نهایی که مقدار پیش‌بینی شده را ارائه می‌دهند. هدف درخت تصمیم این است که داده‌ها را طوری تقسیم کند که نمونه‌های داخل هر گروه تا حد امکان به یکدیگر شبیه باشند. برای این کار از معیار کاهش واریانس (Variance Reduction) استفاده می‌شود.

فرمول

$$Var(s) = \frac{1}{n} \left(\sum_{i=1}^n (y_i - \bar{y}_l)^2 \right)$$

که در آن:

- n تعداد نمونه‌ها
- y_i مقدار واقعی
- $\bar{y}_l$ میانگین مقادیر هدف

معیار انتخاب بهترین تقسیم

درخت تصمیم در هر مرحله تقسیمی را انتخاب می کند که بیشترین کاهش واریانس را ایجاد کند.

$$Gain = Var(parent) - \left(\frac{nL}{n} Var(left) + \frac{nR}{n} Var(right)\right)$$

که در آن:

- Parent : گره فعلی
- Left : شاخه چپ
- Right : شاخه راست

هرچه Gain بزرگ تر باشد، آن تقسیم بهتر است.

تابع پیش بینی

پس از رسیدن به یک برگ، مقدار خروجی برابر میانگین نمونه های موجود در آن برگ خواهد بود:

$$\frac{1}{n} \sum_{i=1}^n y_i$$

مزایا

✓ درک و تفسیر آسان

✓ عدم نیاز به نرمال سازی داده ها

✓ توانایی مدل سازی روابط غیر خطی

✓ قابلیت کار با داده های پیچیده

معایب

× حساسیت بالا به داده‌های آموزشی

× احتمال Overfitting

× ناپایداری در برابر تغییرات کوچک داده‌ها

کد

```
from sklearn.preprocessing import StandardScaler
train_rmse_errors=[]
test_rmse_errors=[]
for d in range(1,5):
    x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=101)
    poly_converter=PolynomialFeatures(degree=d,include_bias=False)
    poly_features=poly_converter.fit_transform(x_train)
    x_test=poly_converter.transform(x_test)
    scaler=StandardScaler()
    poly_features=scaler.fit_transform(poly_features)
    x_test=scaler.transform(x_test)

    model=DecisionTreeRegressor()
    model.fit(poly_features,y_train)

    train_pred=model.predict(poly_features)
    test_pred=model.predict(x_test)

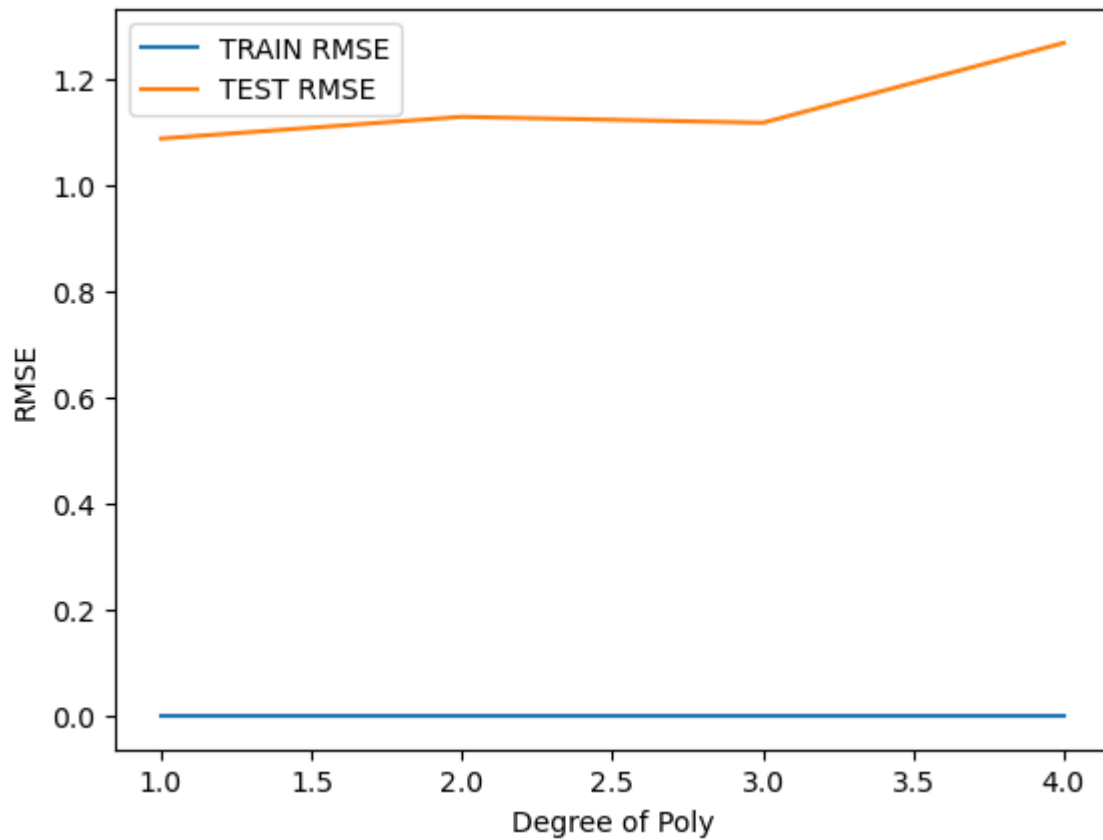
    train_mse=mean_squared_error(y_train,train_pred)
    train_rmse=np.sqrt(train_mse)

    test_mse=mean_squared_error(y_test,test_pred)
    test_rmse=np.sqrt(test_mse)

    train_rmse_errors.append(train_rmse)
    test_rmse_errors.append(test_rmse)
plt.plot(range(1,5),train_rmse_errors[:5],label='TRAIN RMSE')
```

```
plt.plot(range(1,5),test_rmse_errors[:5],label='TEST RMSE')

plt.xlabel('Degree of Poly')
plt.ylabel('RMSE')
plt.legend()
```



درجه يك بهترين خطا رو مي دهد.

كد

```
poly_converter=PolynomialFeatures(degree=2,include_bias=False)
from sklearn.model_selection import GridSearchCV
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=101)
poly_features=poly_converter.fit_transform(x_train)
x_test=poly_converter.transform(x_test)
scaler=StandardScaler()
poly_features=scaler.fit_transform(poly_features)
x_test=scaler.transform(x_test)
from sklearn.tree import DecisionTreeRegressor
```

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8],
    'criterion': ['squared_error', 'absolute_error']
}

tree = DecisionTreeRegressor(random_state=42)

grid = GridSearchCV(
    estimator=tree,
    param_grid=param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

grid.fit(poly_features, y_train)

print("Best Parameters:")
print(grid.best_params_)

print("Best Score:")
print(grid.best_score_)
y_pred=grid.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)

```

Best Parameters:

```
{'criterion': 'squared_error', 'max_depth': 5, 'min_samples_leaf': 4,
'min_samples_split': 20}
```

MAE : 0.8010738959095711

MSE : 1.03550691410886

RMSE : 1.017598601664163

Use Regularization Methods

هدف از Use Regularization Methods

جلوگیری از Overfitting

مفهوم Overfitting

زمانی که مدل داده‌های آموزش را حفظ کند اما روی داده‌های جدید ضعیف عمل کند Overfitting رخ داده است. بیش‌برازش یا Overfitting یکی از مشکلات رایج در یادگیری ماشین است که زمانی رخ می‌دهد که مدل علاوه بر یادگیری الگوهای واقعی موجود در داده‌ها، نویزها و جزئیات غیرضروری داده‌های آموزشی را نیز حفظ کند. در این حالت مدل عملکرد بسیار خوبی روی داده‌های آموزشی دارد، اما هنگام مواجهه با داده‌های جدید و دیده‌نشده، دقت آن به طور قابل توجهی کاهش می‌یابد. به عبارت دیگر، مدل به جای یادگیری الگوهای کلی، داده‌های آموزشی را «حفظ» می‌کند و توانایی تعمیم (Generalization) خود را از دست می‌دهد. به طور معمول، در حالت Overfitting خطای مجموعه آموزشی (Training Error) بسیار کم است، در حالی که خطای مجموعه آزمون (Test Error) مقدار بالایی دارد.

دلایل اصلی ایجاد Overfitting عبارت‌اند از:

- پیچیدگی بیش از حد مدل
- تعداد زیاد ویژگی‌ها نسبت به حجم داده‌ها
- آموزش بیش از حد مدل
- وجود نویز در داده‌ها

برای کاهش Overfitting از روش‌های مختلفی استفاده می‌شود، از جمله:

- افزایش حجم داده‌های آموزشی
- انتخاب ویژگی‌های مهم‌تر
- استفاده از Cross Validation
- محدود کردن پیچیدگی مدل
- استفاده از روش‌های Regularization مانند Ridge و Lasso

در این پروژه، برای جلوگیری از بیش‌برازش از روش‌های Ridge Regression و Lasso Regression استفاده شد. این روش‌ها با اعمال جریمه روی ضرایب مدل، از بزرگ شدن بیش از حد آن‌ها جلوگیری کرده و توانایی تعمیم مدل را افزایش می‌دهند.

Ridge Regression

Ridge نسخه Regularized شده رگرسیون خطی است. در این روش یک جریمه از نوع L2 به تابع خطا اضافه می‌شود.

فرمول:

$$Loss = RSS + \alpha \sum_{j=1}^P \beta_j^2$$

$$Loss = \sum (y_i - \hat{y}_i)^2 + \alpha \sum \beta_j^2$$

ایده اصلی

اگر ضرایب خیلی بزرگ شوند β_j^2

$$\beta_j^2 \setminus \beta_j^2$$

بزرگ می‌شود و مدل جریمه می‌گیرد.

مزایا

- کاهش Overfitting
- مقابله با Multicollinearity
- حفظ تمام ویژگی‌ها

نکته

در Ridge هیچ ضرری دقیقاً صفر نمی‌شود. Ridge Regression با اعمال جریمه روی ضرایب بزرگ، از بیش‌برازش مدل جلوگیری کرده و پایداری آن را افزایش می‌دهد.

کد

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

ridge = Ridge()

param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10, 100]
}

ridge_grid = GridSearchCV(
    ridge,
    param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

ridge_grid.fit(poly_features, y_train)

print(ridge_grid.best_params_)
```

شناسایی بهترین پارامتر

{'alpha': 100}

```
y_pred=ridge_grid.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
```

```
MAE : 0.6997241916211862
MSE : 0.8155351831529175
RMSE : 0.9030698661526236
```

RidgeCV

RidgeCV همان Ridge Regression با انتخاب بهترین α است .

به جای اینکه خودمان مقدار α را تعیین کنیم:

Ridge(alpha=1)

از Cross Validation استفاده می کند. مثال:

```
from sklearn.linear_model import RidgeCV
```

```
ridgecv = RidgeCV(alphas=[0.01,0.1,1,10,100])
```

مدل، بهترین α را پیدا می کند.

$$Loss = RSS + \alpha \sum_{j=1}^P \beta_j^2$$

اما مقدار α توسط Cross Validation انتخاب می شود.

کد

```
from sklearn.linear_model import RidgeCV

ridgecv = RidgeCV(
    alphas=[0.001,0.01,0.1,1,10,100],
    cv=5
)

ridgecv.fit(poly_features, y_train)

print(ridgecv.alpha_)
y_pred=elastic_grid.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
```

100.0

MAE : 0.6405680392934411

MSE : 0.6808326326069908

RMSE : 0.8251258283480106

Lasso Regression

Lasso از جریمه L1 استفاده می کند.

فرمول:

$$Loss = RSS + \alpha \sum |\beta_j|$$

$$Loss = \sum (y_i - \hat{y}_i)^2 + \alpha \sum |\beta_j|$$

برخی ضرایب را صفر می کند.

$$\beta = [2/3.1/8.0/9.0/1.0/02]$$

$$\beta = [2/1.1/6.0/7.0.0]$$

در نتیجه برخی ویژگی ها حذف می شوند.

مزایا

- کاهش Overfitting
- Feature Selection
- مدل ساده تر

کد

```
from sklearn.linear_model import Lasso

lasso = Lasso(max_iter=10000)

param_grid = {
    'alpha': [0.0001, 0.001, 0.01, 0.1, 1]
}

lasso_grid = GridSearchCV(
    lasso,
    param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

lasso_grid.fit(poly_features, y_train)

print(lasso_grid.best_params_)
y_pred=lasso_grid.predict(x_test)
```

```
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
```

```
{'alpha': 0.1}
```

```
MAE : 0.6486056949284813
```

```
MSE : 0.68490044927317
```

LassoCV

LassoCV نسخه خود کار Lasso است. بهترین alpha را با Cross Validation پیدا می کند.

مثال:

```
from sklearn.linear_model import LassoCV
```

```
lassocv = LassoCV(cv=5)
```

$$Loss = RSS + \alpha \sum |\beta_j|$$

اما مقدار α توسط Cross Validation انتخاب می شود.

کد

```
from sklearn.linear_model import LassoCV

lassocv = LassoCV(
    alphas=[0.0001,0.001,0.01,0.1,1],
    cv=5,
    max_iter=10000
)

lassocv.fit(poly_features, y_train)

print(lassocv.alpha_)
y_pred=elastic_grid.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
```

```
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
```

0.1

```
MAE : 0.6405680392934411
MSE : 0.6808326326069908
RMSE : 0.825125828348010
```

ElasticNet

Elastic Net ترکیبی از Ridge و Lasso است.

فرمول:

$$Loss = RSS + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

مزایا

- انتخاب ویژگی مانند Lasso
- پایداری مانند Ridge
- مناسب برای داده‌های دارای ویژگی‌های زیاد

زمانی استفاده می‌شود که

- تعداد ویژگی‌ها زیاد باشد.
- ویژگی‌ها با یکدیگر همبستگی بالا داشته باشند.

کد

```
from sklearn.linear_model import ElasticNet
```

```

elastic = ElasticNet(max_iter=10000)

param_grid = {
    'alpha': [0.0001, 0.001, 0.01, 0.1, 1],
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]
}

elastic_grid = GridSearchCV(
    elastic,
    param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

elastic_grid.fit(poly_features, y_train)

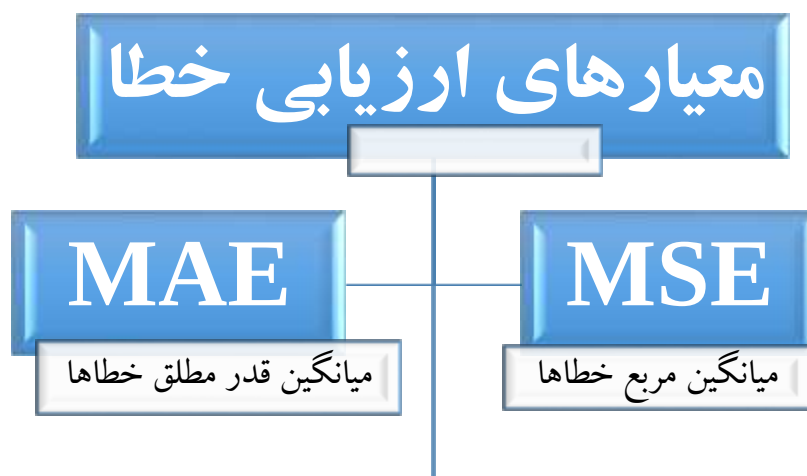
print(elastic_grid.best_params_)
{'alpha': 0.1, 'l1_ratio': 0.9}

y_pred=elastic_grid.predict(x_test)
MAE=mean_absolute_error(y_test,y_pred)
MSE=mean_squared_error(y_test,y_pred)
RMSE=np.sqrt(MSE)
print("MAE : " , MAE)
print("MSE : " , MSE)
print("RMSE : " , RMSE)
MAE : 0.6405680392934411
MSE : 0.6808326326069908
RMSE : 0.8251258283480106

```

Evaluation and Conclusion

معیارهای ارزیابی خطا



نتیجه‌گیری نهایی

مدل‌ها	MAE	MSE	RMSE
Linear Regression	0.649	0.668	0.817
SVR	0.674	0.735	0.857
Decision Tree	0.801	0.801	1.017
Lasso	0.64	0.684	0.82
Ridge	0.699	0.815	0.903
LassoCV	0.640	0.680	0.825
RidgeCV	0.640	0.680	0.825
Elastic Net	0.640	0.680	0.825